

ATIVIDADE_DA_AULA_008_VINICIUS_GIRATO_MORESQUI_RA_26014776-2

Atividade 01:

Python

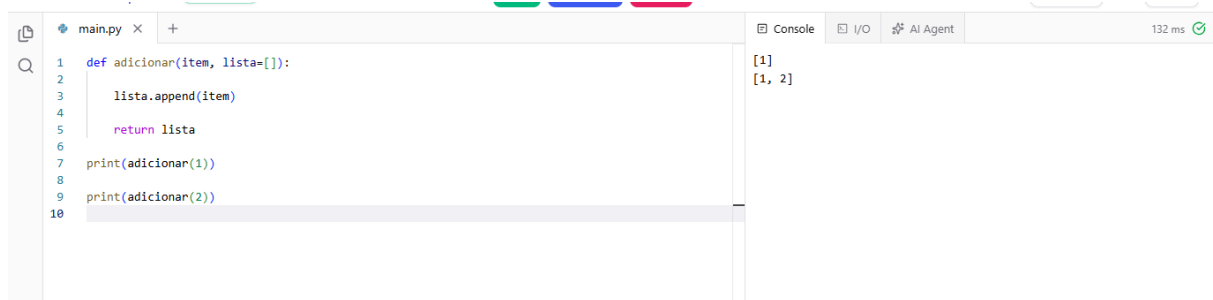
```
def adicionar(item, lista=[]):
```

```
    lista.append(item)
```

```
    return lista
```

```
print(adicionar(1))
```

```
print(adicionar(2))
```



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'adicionar' that takes an 'item' and a default empty list 'lista'. It appends the item to the list and returns it. Below the function definition, there are two print statements: 'print(adicionar(1))' and 'print(adicionar(2))'. To the right of the code editor, a console window displays the output: '[1]' on the first line and '[1, 2]' on the second line. The console also shows a status bar indicating '132 ms' and a green checkmark.

Saída esperada: [1] e [1,2].

Qual conceito da aula explica: Parâmetros com valores padrão mutáveis.

Atividade 02:

2 - Java

```
static void zera(int[] v, int n) {  
    v[0] = 0;  
    n = 0;  
}  
int[] v = {5, 5}; int n = 5;  
zera(v, n);  
System.out.println(v[0] + " " + n);
```

Saída esperada: 0 5.

Qual conceito da aula explica: O Java passa tudo por valor; como o array é um objeto, a cópia da referência permite alterar o estado interno (v[0]), mas a variável do tipo primitivo (int n) é copiada de forma isolada, não afetando o valor original fora do método .

Atividade 03:

```
main.py × +  
1 fs = [lambda: i for i in range(3)]  
2  
3 print([f() for f in fs])  
4
```

Saída esperada: [2 2 2].

Qual conceito da aula explica: Fechamento. As funções anônimas (*lambdas*) capturam a variável de controle do ciclo (i) por referência e não por valor estático no momento da criação.

Atividade 04:

4 • C

```
int contador(void) {  
    static int n = 0;  
    return ++n;  
}  
// em main:  
contador(); contador();  
printf("%d\n", contador());
```

Saída esperada: 3.

Qual conceito da aula explica: Diferente das variáveis locais dinâmicas que vivem apenas na pilha de execução e são destruídas ao fim da função, a variável

declarada como static em C fica armazenada na memória estática, permitindo que a função "lembre" e persista o seu valor entre chamadas sucessivas.

Atividade 05:

5 • Rust

```
fn dobra(v: Vec<i32>) -> Vec<i32> {  
    v.iter().map(|x| x * 2).collect()  
}  
let v = vec![1, 2, 3];  
let d = dobra(v);  
println!("{:?} {:?}", v, d);
```

Saída esperada: O código não compila.

Qual conceito da aula explica: O conceito é a semântica de posse (*ownership*) e movimento (*move*) em Rust. Ao chamar a função, a propriedade do vetor é transferida (*moved*) para o parâmetro interno, impedindo acessos posteriores à variável original para garantir a segurança de memória.

Atividade 06:

6 • Python

```
total = 0  
  
def adiciona(x):  
    total = total + x  
    return total  
  
print(adiciona(5))
```

Saída esperada: O código gera um erro de execução.

Qual conceito da aula explica: os ambientes de referenciamento e escopo de variáveis. Em Python, para modificar ou ler uma variável global dentro de uma

função sem criar um conflito de escopo local, seria necessária a declaração explícita (como a palavra-chave `global`), caso contrário o interpretador trata-a incorretamente como uma nova variável local não inicializada.